Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

<Name1>Rachepalli Nandhitha **<SRN1>PES2UG24S391**

<Name2>Raksha V **<SRN2>PES2UG24CS396**

<SRN3>PES2UG24CS377

<Name3>Priyanka

under the guidance of

Dr. Chithambara

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

<Name1>Rachepalli Nandhitha	**<SRN1>PES2UG24CS391**
<Name2>Raksha V	**<SRN2>PES2UG24CS396**
<Name3>Priyanka	**<SRN3>PES2UG24CS377**

In partial fulfilment for the completion of fourth semester OS Project (UE24CS242B) in the Program of Study Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature
Dr Chithambara
Assistant Professor

DECLARATION

We hereby declare that the **Operating System** has been carried out by us under the guidance of **Dr.Chithambara, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

<name>Rachepalli Nandhitha	**<srn>PES2UG24CS391**	**<Signature>Nandhitha**
<name>Raksha V	**<srn>PES2UG24CS396**	**<Signature>Raksha**
<name>Priyanka	**<srn>PES2UG24CS377**	**<Signature>priyanka**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr.Chithambara, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project implements a lightweight Linux container runtime along with experiments to study operating system behavior. It includes container creation using process isolation techniques, a supervisor system for managing multiple containers, and an inter-process communication mechanism for command handling. A logging system is developed to capture and store container outputs efficiently, while a kernel module is used to monitor and control memory usage. The project also includes scheduler experiments comparing CPU-bound and I/O-bound processes under different priority settings to observe Linux scheduling behavior. Finally, all components are integrated and tested to demonstrate a functional and controlled runtime environment. The results provide practical insights into process management, resource allocation, and scheduling in Linux systems.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	4
3.	METHODOLOGY	8
4.	CODE	12
5.	RESULT AND DISCURSION	18
6.	OUTPUTS	20
7.	GITHUB LINK	21
REFERENCES/BIBLIOGRAPHY		22
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		23

1. INTRODUCTION

The Linux operating system plays a vital role in managing system resources and enabling efficient execution of multiple processes. One of its key responsibilities is to provide process isolation, resource management, and fair scheduling among competing workloads. With the rise of containerization technologies such as Docker, the concept of lightweight virtualization has become increasingly important. Containers allow applications to run in isolated environments while sharing the same underlying operating system, making them efficient and scalable. This project focuses on building a simplified container runtime system and using it as a platform to study important operating system concepts.

The project is divided into multiple tasks that together demonstrate how modern operating systems manage processes and resources. The first task involves creating containers using low-level system calls such as `clone()`. This enables process isolation by using Linux namespaces, allowing each container to have its own independent execution environment. This forms the foundation of the runtime system by ensuring that processes run securely and independently.

The second task introduces a supervisor system along with a command-line interface (CLI). The supervisor acts as a central controller that manages multiple containers, handles their lifecycle, and processes user commands such as start, stop, and list. Communication between the CLI and the supervisor is implemented using inter-process communication (IPC) mechanisms such as pipes or sockets. This demonstrates how different processes coordinate and exchange information in a controlled manner.

In the third task, a logging system is implemented to capture the output generated by containers. This is achieved using pipes and a bounded buffer approach, which ensures that logs are stored efficiently without data loss. The logging system highlights the importance of synchronization and safe data handling when multiple processes produce output simultaneously. It also reflects real-world systems where logs are essential for monitoring and debugging.

The fourth task extends the project into kernel space by implementing a memory monitoring system using a Linux Kernel Module (LKM). This module tracks the memory usage of container processes and enforces predefined limits. It communicates with the user-space runtime through mechanisms such as `ioctl`. This task provides insight into how the operating system kernel interacts with user-space programs and manages critical resources like memory.

The fifth task focuses on scheduler experiments and analysis. In this task, different types of workloads are created, including CPU-bound and I/O-bound processes. These processes are executed concurrently to observe how the Linux scheduler distributes CPU time. By modifying process priorities using nice values and monitoring system behavior using tools like `top`, the project demonstrates how the Completely Fair Scheduler (CFS) ensures fairness and responsiveness. This task bridges theoretical scheduling concepts with practical observations.

Finally, the sixth task involves integrating all components of the project and performing testing. The runtime system is used to run multiple containers, manage resources, collect logs, and analyze scheduling

behavior in a unified environment. This integration ensures that all modules work together correctly and provides a complete understanding of system-level operations.

Overall, this project provides hands-on experience with key operating system concepts such as process creation, inter-process communication, synchronization, memory management, and scheduling. By combining user-space programming with kernel-level components, it offers a comprehensive view of how modern operating systems function. The project not only strengthens theoretical knowledge but also demonstrates its practical application in real-world scenarios.

---

2. PROBLEM DEFINITION

In modern computing systems, multiple processes with different execution behaviors run simultaneously and compete for limited system resources such as CPU time and memory. The Linux operating system must efficiently manage these processes to ensure fairness, responsiveness, and optimal performance. However, understanding how the system internally handles process isolation, communication, resource allocation, and scheduling is not always straightforward without practical experimentation.

The main problem addressed in this project is to design and implement a lightweight container runtime that can manage multiple isolated processes and use it as a platform to study operating system behavior. The project focuses on how containers can be created and controlled, how processes communicate with each other, how logs are managed safely, and how memory usage can be monitored and restricted. Additionally, it aims to analyze how the Linux scheduler allocates CPU time among different types of workloads under varying priority conditions.

Another key challenge is to observe and measure the behavior of CPU-bound and I/O-bound processes when they run concurrently. These processes have different execution patterns, and the scheduler must balance CPU utilization with responsiveness. By modifying parameters such as nice values, the project investigates how priority affects process execution and CPU sharing.

The problem also involves integrating both user-space and kernel-space components. While the user-space runtime is responsible for launching and managing containers, the kernel module is required to monitor and enforce memory limits. Ensuring proper communication and coordination between these components is an important aspect of the system.

The measurable outcomes considered in this project include CPU usage, process responsiveness, and execution behavior under different configurations. The goal is not to develop a new scheduler, but to create controlled experiments that clearly demonstrate how the existing Linux scheduler works in practice.

Overall, this project addresses the challenge of building a simplified runtime environment and using it to analyze key operating system concepts such as process management, resource allocation, and scheduling behavior in a systematic and observable manner.

---

3. METHODOLOGY

The methodology of this project is based on designing a controlled experimental environment using a custom container runtime to observe and analyze Linux operating system behavior. The approach is divided into multiple stages, including runtime setup, workload creation, experiment execution, and data observation.

Initially, a lightweight container runtime is developed in user space. Containers are created using system calls such as clone() with appropriate namespace flags to achieve process isolation. Each container runs independently with its own execution environment. A supervisor process is implemented to manage container lifecycle operations such as start, stop, and monitoring. Communication between the command-line interface and the supervisor is established using inter-process communication (IPC) mechanisms like pipes or sockets.

To capture the output generated by containers, a logging system is implemented using pipes and a bounded buffer approach. This ensures that logs from multiple containers are stored safely without data loss or race conditions. The logging mechanism also helps in analyzing process behavior during execution.

A kernel module is developed to monitor memory usage of container processes. The module tracks memory consumption and enforces predefined soft and hard limits. Communication between the user-space runtime and kernel module is handled using ioctl, enabling controlled interaction between user space and kernel space.

For scheduler analysis, two types of workloads are designed: CPU-bound and I/O-bound processes. The CPU-bound program performs continuous computations without interruption, while the I/O-bound program includes periodic delays using sleep() to simulate waiting behavior. These workloads are executed concurrently within the runtime environment.

Different scheduling configurations are applied by modifying nice values to change process priority. The system is monitored using tools like top to observe CPU usage, priority levels, and execution patterns. Screenshots and logs are collected as evidence of system behavior.

Finally, all components are integrated and tested together. Multiple containers are executed simultaneously to verify correct functionality of process isolation, logging, memory monitoring, and scheduling behavior. The observations from these experiments are used to analyze how the Linux Completely Fair Scheduler distributes CPU time and ensures fairness and responsiveness.

---

4. CODE

This code represents a conceptual implementation of workload control. The actual experiments were conducted using compiled C programs.

Task1,2.3

```
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <pthread.h>
#include <sched.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mount.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
```

```
#include "monitor_ioctl.h"

/*
==========================================================
======== */
#define STACK_SIZE          (1024 * 1024)
#define CONTAINER_ID_LEN      32
#define CONTROL_PATH          "/tmp/mini_runtime.sock"
#define LOG_DIR               "logs"
#define CONTROL_MESSAGE_LEN   512
#define CHILD_COMMAND_LEN     256
#define LOG_CHUNK_SIZE        4096
#define LOG_BUFFER_CAPACITY   16
#define DEFAULT_SOFT_LIMIT    (40UL << 20)
#define DEFAULT_HARD_LIMIT    (64UL << 20)
#define DEVICE_NAME           "container_monitor"

/*
==========================================================
======== */
typedef enum {
   CMD_SUPERVISOR = 0, CMD_START, CMD_RUN, CMD_PS,
CMD_LOGS, CMD_STOP
} command_kind_t;
```

```
typedef enum {
    CONTAINER_STARTING = 0, CONTAINER_RUNNING,
CONTAINER_STOPPED,
    CONTAINER_KILLED, CONTAINER_EXITED
} container_state_t;

typedef struct container_record {
    char                id[CONTAINER_ID_LEN];
    pid_t               host_pid;
    time_t              started_at;
    container_state_t   state;
    unsigned long       soft_limit_bytes;
    unsigned long       hard_limit_bytes;
    int                 exit_code;
    int                 exit_signal;
    int                 stop_requested;
    char                log_path[PATH_MAX];
    struct container_record *next;
} container_record_t;

typedef struct {
    char   container_id[CONTAINER_ID_LEN];
    size_t length;
```

```
    char   data[LOG_CHUNK_SIZE];
} log_item_t;

typedef struct {
    log_item_t      items[LOG_BUFFER_CAPACITY];
    size_t          head, tail, count;
    int             shutting_down;
    pthread_mutex_t mutex;
    pthread_cond_t  not_empty, not_full;
} bounded_buffer_t;

typedef struct {
    command_kind_t kind;
    char           container_id[CONTAINER_ID_LEN];
    char           rootfs[PATH_MAX];
    char           command[CHILD_COMMAND_LEN];
    unsigned long  soft_limit_bytes, hard_limit_bytes;
    int            nice_value;
} control_request_t;

typedef struct {
    int  status;
    char message[CONTROL_MESSAGE_LEN];
    int  run_exit_code;
```

```
    int  run_exit_signal;
} control_response_t;

typedef struct {
    char id[CONTAINER_ID_LEN];
    char rootfs[PATH_MAX];
    char command[CHILD_COMMAND_LEN];
    int  nice_value;
    int  log_write_fd;
} child_config_t;

typedef struct {
    int                 server_fd, monitor_fd;
    volatile int        should_stop;
    pthread_t           logger_thread;
    bounded_buffer_t    log_buffer;
    pthread_mutex_t     metadata_lock;
    container_record_t *containers;
} supervisor_ctx_t;

static supervisor_ctx_t *g_ctx = NULL;
```

```
/*
==================================================================
========
 * Bounded Buffer
 *
==================================================================
======== */
static int bounded_buffer_init(bounded_buffer_t *b)
{
   int rc;
   memset(b, 0, sizeof(*b));
   if ((rc = pthread_mutex_init(&b->mutex, NULL))    != 0) return rc;
   if ((rc = pthread_cond_init(&b->not_empty, NULL)) != 0) {
      pthread_mutex_destroy(&b->mutex); return rc; }
   if ((rc = pthread_cond_init(&b->not_full, NULL))  != 0) {
      pthread_cond_destroy(&b->not_empty);
      pthread_mutex_destroy(&b->mutex); return rc; }
   return 0;
}

static void bounded_buffer_destroy(bounded_buffer_t *b) {
   pthread_cond_destroy(&b->not_full);
   pthread_cond_destroy(&b->not_empty);
   pthread_mutex_destroy(&b->mutex);
```

```
}

static void bounded_buffer_begin_shutdown(bounded_buffer_t *b) {
    pthread_mutex_lock(&b->mutex);
    b->shutting_down = 1;
    pthread_cond_broadcast(&b->not_empty);
    pthread_cond_broadcast(&b->not_full);
    pthread_mutex_unlock(&b->mutex);
}

int bounded_buffer_push(bounded_buffer_t *b, const log_item_t *item)
{
    pthread_mutex_lock(&b->mutex);
    while (b->count == LOG_BUFFER_CAPACITY && !b->shutting_down)
        pthread_cond_wait(&b->not_full, &b->mutex);
    if (b->shutting_down && b->count == LOG_BUFFER_CAPACITY) {
        pthread_mutex_unlock(&b->mutex); return -1; }
    b->items[b->tail] = *item;
    b->tail = (b->tail + 1) % LOG_BUFFER_CAPACITY;
    b->count++;
    pthread_cond_signal(&b->not_empty);
    pthread_mutex_unlock(&b->mutex);
    return 0;
}
```

```
int bounded_buffer_pop(bounded_buffer_t *b, log_item_t *item)
{
    pthread_mutex_lock(&b->mutex);
    while (b->count == 0 && !b->shutting_down)
        pthread_cond_wait(&b->not_empty, &b->mutex);
    if (b->count == 0) {
        pthread_mutex_unlock(&b->mutex); return -1; }
    *item = b->items[b->head];
    b->head = (b->head + 1) % LOG_BUFFER_CAPACITY;
    b->count--;
    pthread_cond_signal(&b->not_full);
    pthread_mutex_unlock(&b->mutex);
    return 0;
}

/*
==================================================================
========
 * Logger consumer thread
 *
==================================================================
======== */
static void *logging_thread(void *arg)
```

```
{
    supervisor_ctx_t *ctx = (supervisor_ctx_t *)arg;
    log_item_t item;
    while (bounded_buffer_pop(&ctx->log_buffer, &item) == 0) {
        char log_path[PATH_MAX] = {0};
        container_record_t *rec;
        pthread_mutex_lock(&ctx->metadata_lock);
        for (rec = ctx->containers; rec; rec = rec->next)
            if (strcmp(rec->id, item.container_id) == 0) {
                strncpy(log_path, rec->log_path, sizeof(log_path)-1);
                break;
            }
        pthread_mutex_unlock(&ctx->metadata_lock);
        if (log_path[0]) {
            int fd = open(log_path, O_WRONLY|O_CREAT|O_APPEND, 0644);
            if (fd >= 0) { write(fd, item.data, item.length); close(fd); }
        }
    }
    return NULL;
}

/*
==================================================================
========
```

```
 * Per-container pipe reader (producer thread)
 *
==========================================================
======== */
typedef struct {
    int             read_fd;
    char            container_id[CONTAINER_ID_LEN];
    bounded_buffer_t *log_buffer;
} reader_args_t;

static void *container_reader_thread(void *arg)
{
    reader_args_t *ra = (reader_args_t *)arg;
    log_item_t item;
    ssize_t n;
    memset(&item, 0, sizeof(item));
    strncpy(item.container_id, ra->container_id, sizeof(item.container_id)-1);
    while ((n = read(ra->read_fd, item.data, sizeof(item.data)-1)) > 0) {
        item.data[n] = '\0';
        item.length  = (size_t)n;
        bounded_buffer_push(ra->log_buffer, &item);
        memset(item.data, 0, sizeof(item.data));
    }
    close(ra->read_fd);
```

```
    free(ra);
    return NULL;
}

/*
================================================================
========
 * Clone child entrypoint
 *
================================================================
======== */
static int child_fn(void *arg)
{
    child_config_t *cfg = (child_config_t *)arg;
    char proc_path[PATH_MAX];
    sethostname(cfg->id, strlen(cfg->id));

    snprintf(proc_path, sizeof(proc_path), "%s/proc", cfg->rootfs);
    mkdir(proc_path, 0555);
    mount("proc",                    proc_path,                    "proc",
MS_NOEXEC|MS_NOSUID|MS_NODEV, NULL);

    if (chroot(cfg->rootfs) != 0) { perror("chroot"); return 1; }
    if (chdir("/") != 0) { perror("chdir"); return 1; }
```

```
    if (cfg->log_write_fd >= 0) {
        dup2(cfg->log_write_fd, STDOUT_FILENO);
        dup2(cfg->log_write_fd, STDERR_FILENO);
        close(cfg->log_write_fd);
    }
    if (cfg->nice_value != 0) nice(cfg->nice_value);

    char *const argv[] = { "/bin/sh", "-c", cfg->command, NULL };
    execv("/bin/sh", argv);
    perror("execv");
    return 1;
}

/*
================================================================
========
 * Monitor ioctl helpers
 *
================================================================
======== */
static int register_with_monitor(int fd, const char *cid, pid_t pid,
                                 unsigned long soft, unsigned long hard)
{
```

```
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid = pid; req.soft_limit_bytes = soft; req.hard_limit_bytes = hard;
    strncpy(req.container_id, cid, sizeof(req.container_id)-1);
    return ioctl(fd, MONITOR_REGISTER, &req) < 0 ? -1 : 0;
}

static int unregister_from_monitor(int fd, const char *cid, pid_t pid)
{
    struct monitor_request req;
    memset(&req, 0, sizeof(req));
    req.pid = pid;
    strncpy(req.container_id, cid, sizeof(req.container_id)-1);
    return ioctl(fd, MONITOR_UNREGISTER, &req) < 0 ? -1 : 0;
}

/*
==================================================================
========
 * Signal handlers
 *
==================================================================
======== */
static void sigchld_handler(int sig)
```

```
{
    (void)sig;
    int saved = errno;
    int wstatus;
    pid_t pid;
    while ((pid = waitpid(-1, &wstatus, WNOHANG)) > 0) {
        if (!g_ctx) continue;
        container_record_t *rec;
        pthread_mutex_lock(&g_ctx->metadata_lock);
        for (rec = g_ctx->containers; rec; rec = rec->next) {
            if (rec->host_pid != pid) continue;
            if (WIFEXITED(wstatus)) {
                rec->exit_code   = WEXITSTATUS(wstatus);
                rec->exit_signal = 0;
                rec->state = CONTAINER_EXITED;
            } else if (WIFSIGNALED(wstatus)) {
                rec->exit_signal = WTERMSIG(wstatus);
                rec->exit_code   = 0;
                /* Attribution rule from project guide §Task 4 */
                if (rec->stop_requested)
                    rec->state = CONTAINER_STOPPED;
                else if (rec->exit_signal == SIGKILL)
                    rec->state = CONTAINER_KILLED;
                else
```

```
                rec->state = CONTAINER_STOPPED;
            }
            if (g_ctx->monitor_fd >= 0)
                unregister_from_monitor(g_ctx->monitor_fd,
                                        rec->id, rec->host_pid);
            break;
        }
        pthread_mutex_unlock(&g_ctx->metadata_lock);
    }
    errno = saved;
}

static void shutdown_handler(int sig)
{
    (void)sig;
    if (g_ctx) g_ctx->should_stop = 1;
}

/*
==================================================================
========
 * Launch a container
```

```
 *
==========================================================================
======== */
static int launch_container(supervisor_ctx_t *ctx,
                            const control_request_t *req)
{
    container_record_t *existing;
    pthread_mutex_lock(&ctx->metadata_lock);
    for (existing = ctx->containers; existing; existing = existing->next)
        if (strcmp(existing->id, req->container_id) == 0 &&
            existing->state == CONTAINER_RUNNING) {
            pthread_mutex_unlock(&ctx->metadata_lock);
            fprintf(stderr, "Container '%s' already running\n", req->container_id);
            return -1;
        }
    pthread_mutex_unlock(&ctx->metadata_lock);

    int pipefd[2];
    if (pipe(pipefd) != 0) { perror("pipe"); return -1; }

    char *stack = malloc(STACK_SIZE);
    if (!stack) { close(pipefd[0]); close(pipefd[1]); return -1; }

    child_config_t *cfg = malloc(sizeof(*cfg));
```

```
    if (!cfg) { free(stack); close(pipefd[0]); close(pipefd[1]); return -1; }

    strncpy(cfg->id,      req->container_id, sizeof(cfg->id)-1);
    strncpy(cfg->rootfs,  req->rootfs,       sizeof(cfg->rootfs)-1);
    strncpy(cfg->command, req->command,      sizeof(cfg->command)-1);
    cfg->nice_value   = req->nice_value;
    cfg->log_write_fd = pipefd[1];

    pid_t pid = clone(child_fn, stack + STACK_SIZE,

CLONE_NEWPID|CLONE_NEWUTS|CLONE_NEWNS|SIGCHLD, cfg);
    close(pipefd[1]);
    free(stack);

    if (pid < 0) {
        perror("clone"); free(cfg); close(pipefd[0]); return -1; }

    container_record_t *rec = calloc(1, sizeof(*rec));
    if (!rec) { close(pipefd[0]); free(cfg); return -1; }

    strncpy(rec->id, req->container_id, sizeof(rec->id)-1);
    rec->host_pid         = pid;
    rec->started_at       = time(NULL);
    rec->state            = CONTAINER_RUNNING;
```

```
rec->soft_limit_bytes = req->soft_limit_bytes;
rec->hard_limit_bytes = req->hard_limit_bytes;
rec->stop_requested   = 0;

mkdir(LOG_DIR, 0755);
snprintf(rec->log_path, sizeof(rec->log_path),
        "%s/%s.log", LOG_DIR, rec->id);

pthread_mutex_lock(&ctx->metadata_lock);
rec->next = ctx->containers;
ctx->containers = rec;
pthread_mutex_unlock(&ctx->metadata_lock);

reader_args_t *ra = malloc(sizeof(*ra));
if (ra) {
    ra->read_fd    = pipefd[0];
    ra->log_buffer = &ctx->log_buffer;
    strncpy(ra->container_id, req->container_id,
            sizeof(ra->container_id)-1);
    pthread_t rt;
    if (pthread_create(&rt, NULL, container_reader_thread, ra) == 0)
        pthread_detach(rt);
    else { free(ra); close(pipefd[0]); }
} else close(pipefd[0]);
```

```
    if (ctx->monitor_fd >= 0)
        register_with_monitor(ctx->monitor_fd, rec->id, pid,
                              rec->soft_limit_bytes, rec->hard_limit_bytes);

    free(cfg);
    fprintf(stderr, "[supervisor] started '%s' pid=%d\n", rec->id, pid);
    return 0;
}

/*
============================================================
========
 * Handle one control client
 *
============================================================
======== */
static int handle_client(supervisor_ctx_t *ctx, int cfd)
{
    control_request_t  req;
    control_response_t resp;
    memset(&resp, 0, sizeof(resp));

    if (recv(cfd, &req, sizeof(req), 0) != (ssize_t)sizeof(req)) {
```

```
    resp.status = -1;
    snprintf(resp.message, sizeof(resp.message), "bad request");
    send(cfd, &resp, sizeof(resp), 0);
    return -1;
}

switch (req.kind) {

case CMD_START: {
    int rc = launch_container(ctx, &req);
    resp.status = rc;
    snprintf(resp.message, sizeof(resp.message),
            rc == 0 ? "ok" : "launch failed");
    send(cfd, &resp, sizeof(resp), 0);
    break;
}

case CMD_RUN: {
    int rc = launch_container(ctx, &req);
    if (rc != 0) {
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "launch failed");
        send(cfd, &resp, sizeof(resp), 0);
        break;
```

```
    }
    /* Acknowledge start */
    resp.status = 0;
    snprintf(resp.message, sizeof(resp.message), "running");
    send(cfd, &resp, sizeof(resp), 0);

    /* Wait until container exits (poll; SIGCHLD updates state) */
    container_state_t st = CONTAINER_RUNNING;
    while (st == CONTAINER_RUNNING || st == CONTAINER_STARTING) {
        struct timespec ts = {0, 100000000L};
        nanosleep(&ts, NULL);
        container_record_t *rec;
        pthread_mutex_lock(&ctx->metadata_lock);
        for (rec = ctx->containers; rec; rec = rec->next)
            if (strcmp(rec->id, req.container_id) == 0) {
                st = rec->state; break; }
        pthread_mutex_unlock(&ctx->metadata_lock);
    }

    /* Send final status */
    memset(&resp, 0, sizeof(resp));
    container_record_t *rec;
    pthread_mutex_lock(&ctx->metadata_lock);
```

```
    for (rec = ctx->containers; rec; rec = rec->next)
        if (strcmp(rec->id, req.container_id) == 0) {
            resp.status         = 0;
            resp.run_exit_code   = rec->exit_code;
            resp.run_exit_signal = rec->exit_signal;
            snprintf(resp.message, sizeof(resp.message),
                     "state=%s code=%d signal=%d",
                     state_to_string(rec->state),
                     rec->exit_code, rec->exit_signal);
            break;
        }
    pthread_mutex_unlock(&ctx->metadata_lock);
    send(cfd, &resp, sizeof(resp), 0);
    break;
}

case CMD_PS: {
    char *p = resp.message, *end = resp.message + sizeof(resp.message) - 1;
    container_record_t *rec;
    pthread_mutex_lock(&ctx->metadata_lock);
    for (rec = ctx->containers; rec && p < end; rec = rec->next) {
        char ts[32];
        strftime(ts, sizeof(ts), "%H:%M:%S",
                 localtime(&rec->started_at));
```

```
        int w = snprintf(p, (size_t)(end-p),
            "%-16s pid=%-6d state=%-8s soft=%3luMiB hard=%3luMiB "
            "code=%-3d sig=%-3d start=%s\n",
            rec->id, rec->host_pid, state_to_string(rec->state),
            rec->soft_limit_bytes>>20, rec->hard_limit_bytes>>20,
            rec->exit_code, rec->exit_signal, ts);
        if (w > 0) p += w;
    }
    pthread_mutex_unlock(&ctx->metadata_lock);
    resp.status = 0;
    if (p == resp.message)
        snprintf(resp.message, sizeof(resp.message), "(no containers)\n");
    send(cfd, &resp, sizeof(resp), 0);
    break;
}

case CMD_LOGS: {
    int found = 0;
    container_record_t *rec;
    pthread_mutex_lock(&ctx->metadata_lock);
    for (rec = ctx->containers; rec; rec = rec->next)
        if (strcmp(rec->id, req.container_id) == 0) {
            snprintf(resp.message, sizeof(resp.message), "%s", rec->log_path);
            found = 1; break;
```

```
        }
    pthread_mutex_unlock(&ctx->metadata_lock);
    resp.status = found ? 0 : -1;
    if (!found) snprintf(resp.message, sizeof(resp.message),
                        "container '%s' not found", req.container_id);
    send(cfd, &resp, sizeof(resp), 0);
    break;
}

case CMD_STOP: {
    int found = 0;
    container_record_t *rec;
    pthread_mutex_lock(&ctx->metadata_lock);
    for (rec = ctx->containers; rec; rec = rec->next)
        if (strcmp(rec->id, req.container_id) == 0 &&
            rec->state == CONTAINER_RUNNING) {
            /* MUST set stop_requested before signal (attribution rule) */
            rec->stop_requested = 1;
            kill(rec->host_pid, SIGTERM);
            found = 1; break;
        }
    pthread_mutex_unlock(&ctx->metadata_lock);
    resp.status = found ? 0 : -1;
    snprintf(resp.message, sizeof(resp.message),
```

```
            found ? "stop signal sent"
                  : "container not found or not running");
        send(cfd, &resp, sizeof(resp), 0);
        break;
    }

    default:
        resp.status = -1;
        snprintf(resp.message, sizeof(resp.message), "unknown command");
        send(cfd, &resp, sizeof(resp), 0);
        break;
    }
    return 0;
}

/*
================================================================
========
 * Supervisor daemon
 *
================================================================
======== */
static int run_supervisor(const char *rootfs)
{
```

```
supervisor_ctx_t ctx;
int rc;
memset(&ctx, 0, sizeof(ctx));
ctx.server_fd = ctx.monitor_fd = -1;
g_ctx = &ctx;

if ((rc = pthread_mutex_init(&ctx.metadata_lock, NULL)) != 0) {
    errno = rc; perror("mutex_init"); return 1; }
if ((rc = bounded_buffer_init(&ctx.log_buffer)) != 0) {
    errno = rc; perror("bb_init"); goto out_mutex; }

ctx.monitor_fd = open("/dev/" DEVICE_NAME, O_RDWR);
if (ctx.monitor_fd < 0)
    fprintf(stderr, "[supervisor] WARNING: /dev/%s unavailable – "
                "memory monitoring disabled\n", DEVICE_NAME);

unlink(CONTROL_PATH);
ctx.server_fd = socket(AF_UNIX, SOCK_STREAM, 0);
if (ctx.server_fd < 0) { perror("socket"); goto out_bb; }
{
    struct sockaddr_un addr;
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path)-1);
```

```
    if (bind(ctx.server_fd, (struct sockaddr*)&addr, sizeof(addr)) < 0)
        { perror("bind"); goto out_sock; }
    if (listen(ctx.server_fd, 8) < 0)
        { perror("listen"); goto out_sock; }
}

{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sigchld_handler;
    sa.sa_flags   = SA_RESTART|SA_NOCLDSTOP;
    sigaction(SIGCHLD, &sa, NULL);
    sa.sa_handler = shutdown_handler;
    sa.sa_flags   = SA_RESTART;
    sigaction(SIGINT,  &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
}

if ((rc = pthread_create(&ctx.logger_thread, NULL,
                         logging_thread, &ctx)) != 0) {
    errno = rc; perror("logger thread"); goto out_sock; }

fprintf(stderr, "[supervisor] ready (rootfs=%s socket=%s)\n",
        rootfs, CONTROL_PATH);
```

```
while (!ctx.should_stop) {
    fd_set rfds;
    struct timeval tv = {1, 0};
    FD_ZERO(&rfds); FD_SET(ctx.server_fd, &rfds);
    int sel = select(ctx.server_fd+1, &rfds, NULL, NULL, &tv);
    if (sel < 0)  { if (errno == EINTR) continue; perror("select"); break; }
    if (sel == 0) continue;
    int cfd = accept(ctx.server_fd, NULL, NULL);
    if (cfd < 0)  { if (errno == EINTR) continue; perror("accept"); continue; }
    handle_client(&ctx, cfd);
    close(cfd);
}

fprintf(stderr, "[supervisor] shutting down\n");
{
    container_record_t *rec;
    pthread_mutex_lock(&ctx.metadata_lock);
    for (rec = ctx.containers; rec; rec = rec->next)
        if (rec->state == CONTAINER_RUNNING) {
            rec->stop_requested = 1;
            kill(rec->host_pid, SIGTERM);
            if (ctx.monitor_fd >= 0)
                unregister_from_monitor(ctx.monitor_fd,
```

```
                                rec->id, rec->host_pid);
            }
        pthread_mutex_unlock(&ctx.metadata_lock);
    }
    bounded_buffer_begin_shutdown(&ctx.log_buffer);
    pthread_join(ctx.logger_thread, NULL);

out_sock:
    close(ctx.server_fd);
    unlink(CONTROL_PATH);
out_bb:
    bounded_buffer_destroy(&ctx.log_buffer);
    if (ctx.monitor_fd >= 0) close(ctx.monitor_fd);
    { container_record_t *r = ctx.containers;
      while (r) { container_record_t *n = r->next; free(r); r = n; } }
out_mutex:
    pthread_mutex_destroy(&ctx.metadata_lock);
    return 0;
}

/*
==================================================================
========
 * Client: connect and exchange one request/response
```

```
 *
==================================================================
======== */
static int send_control_request(const control_request_t *req)
{
    int fd;
    struct sockaddr_un addr;
    control_response_t resp;

    fd = socket(AF_UNIX, SOCK_STREAM, 0);
    if (fd < 0) { perror("socket"); return 1; }
    memset(&addr, 0, sizeof(addr));
    addr.sun_family = AF_UNIX;
    strncpy(addr.sun_path, CONTROL_PATH, sizeof(addr.sun_path)-1);
    if (connect(fd, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("connect (is the supervisor running?)");
        close(fd); return 1; }
    if (send(fd, req, sizeof(*req), 0) != (ssize_t)sizeof(*req)) {
        perror("send"); close(fd); return 1; }

    if (req->kind == CMD_RUN) {
        /* First recv: launch ack */
        if (recv(fd, &resp, sizeof(resp), 0) != (ssize_t)sizeof(resp)) {
            perror("recv ack"); close(fd); return 1; }
```

```
    if (resp.status != 0) {
        fprintf(stderr, "run failed: %s\n", resp.message);
        close(fd); return 1; }
    fprintf(stderr, "[run] waiting for container to finish…\n");
    /* Second recv: final exit (blocks until container done) */
    if (recv(fd, &resp, sizeof(resp), 0) != (ssize_t)sizeof(resp)) {
        perror("recv final"); close(fd); return 1; }
    close(fd);
    printf("%s\n", resp.message);
    return resp.run_exit_signal ? 128 + resp.run_exit_signal
                                : resp.run_exit_code;
}

if (recv(fd, &resp, sizeof(resp), 0) != (ssize_t)sizeof(resp)) {
    perror("recv"); close(fd); return 1; }
close(fd);

if (req->kind == CMD_LOGS && resp.status == 0) {
    FILE *f = fopen(resp.message, "r");
    if (f) { char buf[512];
            while (fgets(buf, sizeof(buf), f)) fputs(buf, stdout);
            fclose(f); }
    else fprintf(stderr, "Cannot open log: %s\n", resp.message);
    return 0;
```

```
    }
    printf("%s\n", resp.message);
    return resp.status == 0 ? 0 : 1;
}

/*
==================================================================
========
 * CLI helpers
 *
==================================================================
======== */
static int parse_mib_flag(const char *flag, const char *value, unsigned long
*out)
{
    char *end; unsigned long mib;
    errno = 0; mib = strtoul(value, &end, 10);
    if (errno || end == value || *end || mib > ULONG_MAX/(1UL<<20)) {
        fprintf(stderr, "Invalid value for %s: %s\n", flag, value); return -1; }
    *out = mib << 20; return 0;
}

static int parse_optional_flags(control_request_t *req,
                                int argc, char *argv[], int start)
```

```
{
   int i;
   for (i = start; i < argc; i += 2) {
      if (i+1 >= argc) { fprintf(stderr,"Missing value for %s\n",argv[i]); return -
1; }
      if (!strcmp(argv[i],"--soft-mib")) {
         if        (parse_mib_flag("--soft-mib",argv[i+1],&req->soft_limit_bytes))
return -1;
      } else if (!strcmp(argv[i],"--hard-mib")) {
         if      (parse_mib_flag("--hard-mib",argv[i+1],&req->hard_limit_bytes))
return -1;
      } else if (!strcmp(argv[i],"--nice")) {
         char *end; long nv = strtol(argv[i+1],&end,10);
         if (errno||end==argv[i+1]||*end||nv<-20||nv>19) {
            fprintf(stderr,"Invalid --nice: %s\n",argv[i+1]); return -1; }
         req->nice_value = (int)nv;
      } else { fprintf(stderr,"Unknown option: %s\n",argv[i]); return -1; }
   }
   if (req->soft_limit_bytes > req->hard_limit_bytes) {
      fprintf(stderr,"soft limit cannot exceed hard limit\n"); return -1; }
   return 0;
}

static void usage(const char *p) {
```

```
    fprintf(stderr,
        "Usage:\n"
        "  %s supervisor <base-rootfs>\n"
        "  %s start <id> <rootfs> <cmd> [--soft-mib N] [--hard-mib N] [--nice N]\n"
        "  %s run   <id> <rootfs> <cmd> [--soft-mib N] [--hard-mib N] [--nice N]\n"
        "  %s ps\n  %s logs <id>\n  %s stop <id>\n",
        p,p,p,p,p,p);
}

static int cmd_start(int argc, char *argv[]) {
    control_request_t req;
    if (argc < 5) { usage(argv[0]); return 1; }
    memset(&req,0,sizeof(req)); req.kind=CMD_START;
    strncpy(req.container_id,argv[2],sizeof(req.container_id)-1);
    strncpy(req.rootfs,argv[3],sizeof(req.rootfs)-1);
    strncpy(req.command,argv[4],sizeof(req.command)-1);
    req.soft_limit_bytes=DEFAULT_SOFT_LIMIT;
req.hard_limit_bytes=DEFAULT_HARD_LIMIT;
    if (parse_optional_flags(&req,argc,argv,5)) return 1;
    return send_control_request(&req);
}

static int cmd_run(int argc, char *argv[]) {
    control_request_t req;
```

```
    if (argc < 5) { usage(argv[0]); return 1; }
    memset(&req,0,sizeof(req)); req.kind=CMD_RUN;
    strncpy(req.container_id,argv[2],sizeof(req.container_id)-1);
    strncpy(req.rootfs,argv[3],sizeof(req.rootfs)-1);
    strncpy(req.command,argv[4],sizeof(req.command)-1);
    req.soft_limit_bytes=DEFAULT_SOFT_LIMIT;
req.hard_limit_bytes=DEFAULT_HARD_LIMIT;
    if (parse_optional_flags(&req,argc,argv,5)) return 1;
    return send_control_request(&req);
}

static int cmd_ps(void) {
    control_request_t req; memset(&req,0,sizeof(req)); req.kind=CMD_PS;
    return send_control_request(&req);
}

static int cmd_logs(int argc, char *argv[]) {
    if (argc < 3) { fprintf(stderr,"Usage: %s logs <id>\n",argv[0]); return 1; }
    control_request_t req; memset(&req,0,sizeof(req)); req.kind=CMD_LOGS;
    strncpy(req.container_id,argv[2],sizeof(req.container_id)-1);
    return send_control_request(&req);
}

static int cmd_stop(int argc, char *argv[]) {
```

```
    if (argc < 3) { fprintf(stderr,"Usage: %s stop <id>\n",argv[0]); return 1; }
    control_request_t req; memset(&req,0,sizeof(req)); req.kind=CMD_STOP;
    strncpy(req.container_id,argv[2],sizeof(req.container_id)-1);
    return send_control_request(&req);
}

/*
==================================================================
========
 * main
 *
==================================================================
======== */
int main(int argc, char *argv[])
{
    if (argc < 2) { usage(argv[0]); return 1; }
    if (!strcmp(argv[1],"supervisor")) {
        if (argc < 3) { fprintf(stderr,"Usage: %s supervisor <rootfs>\n",argv[0]);
return 1; }
        return run_supervisor(argv[2]);
    }
    if (!strcmp(argv[1],"start")) return cmd_start(argc,argv);
    if (!strcmp(argv[1],"run"))   return cmd_run(argc,argv);
    if (!strcmp(argv[1],"ps"))    return cmd_ps();
```

```
    if (!strcmp(argv[1],"logs"))  return cmd_logs(argc,argv);
    if (!strcmp(argv[1],"stop"))  return cmd_stop(argc,argv);
    usage(argv[0]); return 1;
}
```

Task4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

static size_t parse_size_mb(const char *arg, size_t fallback)
{
    char *end = NULL;
    unsigned long value = strtoul(arg, &end, 10);

    if (!arg || *arg == '\0' || (end && *end != '\0') || value == 0)
        return fallback;
    return (size_t)value;
}

static useconds_t parse_sleep_ms(const char *arg, useconds_t fallback)
{
    char *end = NULL;
    unsigned long value = strtoul(arg, &end, 10);

    if (!arg || *arg == '\0' || (end && *end != '\0'))
        return fallback;
    return (useconds_t)(value * 1000U);
}

int main(int argc, char *argv[])
{
    const size_t chunk_mb = (argc > 1) ? parse_size_mb(argv[1], 8) : 8;
```

```
    const useconds_t sleep_us = (argc > 2) ? parse_sleep_ms(argv[2], 1000U) :
1000U * 1000U;
    const size_t chunk_bytes = chunk_mb * 1024U * 1024U;
    int count = 0;

    while (1) {
        char *mem = malloc(chunk_bytes);
        if (!mem) {
            printf("malloc failed after %d allocations\n", count);
            break;
        }

        memset(mem, 'A', chunk_bytes);
        count++;
        printf("allocation=%d chunk=%zuMB total=%zuMB\n",
               count, chunk_mb, (size_t)count * chunk_mb);
        fflush(stdout);
        usleep(sleep_us);
    }

    return 0;
}
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/fs.h>
#include <linux/kernel.h>
#include <linux/list.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/pid.h>
#include <linux/sched/signal.h>
#include <linux/slab.h>
#include <linux/timer.h>
#include <linux/uaccess.h>
#include <linux/version.h>
```

```
#include "monitor_ioctl.h"

#define DEVICE_NAME "container_monitor"
#define CHECK_INTERVAL_SEC 1

static long get_rss_bytes(pid_t pid)
{
    struct task_struct *task;
    struct mm_struct *mm;
    long rss_pages = 0;

    rcu_read_lock();
    task = pid_task(find_vpid(pid), PIDTYPE_PID);
    if (!task) {
        rcu_read_unlock();
        return -1;
    }
    get_task_struct(task);
    rcu_read_unlock();

    mm = get_task_mm(task);
    if (mm) {
        rss_pages = get_mm_rss(mm);
        mmput(mm);
    }
    put_task_struct(task);

    return rss_pages * PAGE_SIZE;
}

static void log_soft_limit_event(const char *container_id,
                                 pid_t pid,
                                 unsigned long limit_bytes,
                                 long rss_bytes)
{
```

```
   printk(KERN_WARNING
        "[container_monitor] SOFT LIMIT container=%s pid=%d rss=%ld
limit=%lu\n",
        container_id, pid, rss_bytes, limit_bytes);
}

static void kill_process(const char *container_id,
                    pid_t pid,
                    unsigned long limit_bytes,
                    long rss_bytes)
{
   struct task_struct *task;

   rcu_read_lock();
   task = pid_task(find_vpid(pid), PIDTYPE_PID);
   if (task)
      send_sig(SIGKILL, task, 1);
   rcu_read_unlock();

   printk(KERN_WARNING
        "[container_monitor] HARD LIMIT container=%s pid=%d rss=%ld
limit=%lu\n",
        container_id, pid, rss_bytes, limit_bytes);
}


   mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);
}

static long monitor_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
   struct monitor_request req;

   (void)f;

   if (cmd != MONITOR_REGISTER && cmd != MONITOR_UNREGISTER)
```

```
        return -EINVAL;

    if (copy_from_user(&req, (struct monitor_request __user *)arg, sizeof(req)))
        return -EFAULT;

    if (cmd == MONITOR_REGISTER) {
        printk(KERN_INFO
                "[container_monitor] Registering container=%s pid=%d soft=%lu
hard=%lu\n",
                req.container_id, req.pid, req.soft_limit_bytes, req.hard_limit_bytes);



        return 0;
    }

    printk(KERN_INFO
            "[container_monitor] Unregister request container=%s pid=%d\n",
            req.container_id, req.pid);


    return -ENOENT;
}

/* --- Provided: file operations --- */
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = monitor_ioctl,
};

/* --- Provided: Module Init --- */
static int __init monitor_init(void)
{
    if (alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME) < 0)
        return -1;
```

```
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)
    cl = class_create(DEVICE_NAME);
#else
    cl = class_create(THIS_MODULE, DEVICE_NAME);
#endif
    if (IS_ERR(cl)) {
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(cl);
    }

    if (IS_ERR(device_create(cl, NULL, dev_num, NULL, DEVICE_NAME)))
{
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }

    cdev_init(&c_dev, &fops);
    if (cdev_add(&c_dev, dev_num, 1) < 0) {
        device_destroy(cl, dev_num);
        class_destroy(cl);
        unregister_chrdev_region(dev_num, 1);
        return -1;
    }

    timer_setup(&monitor_timer, timer_callback, 0);
    mod_timer(&monitor_timer, jiffies + CHECK_INTERVAL_SEC * HZ);

    printk(KERN_INFO "[container_monitor] Module loaded. Device:
/dev/%s\n", DEVICE_NAME);
    return 0;
}


    cdev_del(&c_dev);
    device_destroy(cl, dev_num);
```

```
    class_destroy(cl);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "[container_monitor] Module unloaded.\n");
}

module_init(monitor_init);
module_exit(monitor_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Supervised multi-container memory monitor");

#ifndef MONITOR_IOCTL_H
#define MONITOR_IOCTL_H

#ifdef __KERNEL__
#include <linux/ioctl.h>
#include <linux/types.h>
#else
#include <sys/ioctl.h>
#include <sys/types.h>
#endif

#define MONITOR_NAME_LEN 32

struct monitor_request {
    pid_t pid;
    unsigned long soft_limit_bytes;
    unsigned long hard_limit_bytes;
    char container_id[MONITOR_NAME_LEN];
};

#define MONITOR_MAGIC 'M'
#define MONITOR_REGISTER _IOW(MONITOR_MAGIC, 1, struct monitor_request)
```

```
#define MONITOR_UNREGISTER _IOW(MONITOR_MAGIC, 2, struct monitor_request)

#endif
```

Task 5

Cpu.c

```
#include

<stdio.h> int

main() { long long

i = 0;

while i++;

if (i % 100000000 == 0) {  printf("CPU

running: %lld\n",i);

}  return

0;

  }
```

Io.c

```
  #include <stdio.h>

  #include <unistd.h>


  int main() { while

  (1) {

printf("IO process running...\n"); sleep(1);
```

```
  }  return
  0;
}
```

Task 6

```
obj-m += monitor.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)
# If you want host-built workload binaries to run directly inside an Alpine
# rootfs, you can override this with WORKLOAD_LDFLAGS=-static when your
# toolchain supports it.
WORKLOAD_LDFLAGS ?= -static

USER_TARGETS := engine memory_hog cpu_hog io_pulse

all: $(USER_TARGETS) module

ci: WORKLOAD_LDFLAGS =
ci: $(USER_TARGETS)

module: monitor.ko

engine: engine.c monitor_ioctl.h
gcc -O2 -Wall -Wextra -o engine engine.c -lpthread
```

```
memory_hog: memory_hog.c
gcc -O2 -Wall $(WORKLOAD_LDFLAGS) -o memory_hog memory_hog.c

cpu_hog: cpu_hog.c
gcc -O2 -Wall $(WORKLOAD_LDFLAGS) -o cpu_hog cpu_hog.c

io_pulse: io_pulse.c
gcc -O2 -Wall $(WORKLOAD_LDFLAGS) -o io_pulse io_pulse.c

monitor.ko: monitor.c monitor_ioctl.h
$(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
if [ -d "$(KDIR)" ]; then $(MAKE) -C $(KDIR) M=$(PWD) clean; fi
rm -f $(USER_TARGETS) *.o *.mod *.mod.c *.symvers *.order
rm -f *.log
rm -rf logs
rm -f /tmp/mini_runtime.sock

.PHONY: all ci module clean
```

```
#!/usr/bin/env bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$SCRIPT_DIR"
```

```
BUILD_ONLY=0
if [[ "${1:-}" == "--build-only" ]]; then
    BUILD_ONLY=1
fi

ok() {
    echo "[OK] $1"
}

warn() {
    echo "[WARN] $1"
}

fail() {
    echo "[FAIL] $1" >&2
    exit 1
}

echo "== Supervised Runtime Project Preflight =="

if [[ -r /etc/os-release ]]; then
    # shellcheck disable=SC1091
    source /etc/os-release
    [[ "${ID:-}" == "ubuntu" ]] || fail "Unsupported distro: ${ID:-unknown}. Use Ubuntu 22.04/24.04 VM."
```

```
    case "${VERSION_ID:-}" in
        22.04|24.04) ok "Ubuntu version ${VERSION_ID} detected." ;;
        *) fail "Unsupported Ubuntu version: ${VERSION_ID:-unknown}. Use 22.04 or 24.04." ;;
    esac
else
    fail "Cannot read /etc/os-release to verify environment."
fi

if grep -qi microsoft /proc/sys/kernel/osrelease 2>/dev/null || grep -qi microsoft /proc/version 2>/dev/null; then
    fail "WSL detected. This project does not support WSL."
fi
ok "No WSL signature detected."

if command -v systemd-detect-virt >/dev/null 2>&1; then
    VIRT="$(systemd-detect-virt || true)"
    [[ "$VIRT" != "none" ]] || fail "No VM detected. Use an Ubuntu VM for this project."
    ok "Virtualized environment detected: $VIRT."
else
    warn "systemd-detect-virt not found; cannot strictly verify VM."
fi

if command -v mokutil >/dev/null 2>&1; then
    SB_STATE="$(mokutil --sb-state 2>/dev/null || true)"
    if echo "$SB_STATE" | grep -qi "SecureBoot enabled"; then
```

```
        fail "Secure Boot is enabled. Disable it before module testing."
    fi
    if echo "$SB_STATE" | grep -qi "SecureBoot disabled"; then
        ok "Secure Boot is disabled."
    else
        warn "Could not determine Secure Boot state via mokutil output."
    fi
else
    warn "mokutil not installed; cannot auto-check Secure Boot state."
fi

KBUILD_DIR="/lib/modules/$(uname -r)/build"
[[ -d "$KBUILD_DIR" ]] || fail "Kernel headers missing for $(uname -r). Install linux-headers-$(uname -r)."
ok "Kernel headers found at $KBUILD_DIR."

echo "Building user targets and module..."
LOG_FILE=$(mktemp /tmp/runtime_preflight_make.XXXXXX.log)
if make all >"$LOG_FILE" 2>&1; then
    ok "Boilerplate build succeeded."
else
    tail -n 20 "$LOG_FILE" || true
    fail "Build failed. See $LOG_FILE for details."
fi
rm -f "$LOG_FILE"
```

```
if [[ "$BUILD_ONLY" -eq 1 ]]; then
    ok "Build-only mode complete. Skipped insmod/rmmod and /dev checks."
    exit 0
fi

[[ "$(id -u)" -eq 0 ]] || fail "Run as root (sudo ./environment-check.sh) to verify
insmod/rmmod and /dev/container_monitor."

INSERTED_BY_SCRIPT=0
cleanup() {
    if [[ "$INSERTED_BY_SCRIPT" -eq 1 ]]; then
        rmmod monitor >/dev/null 2>&1 || true
    fi
}
trap cleanup EXIT

if lsmod | awk '{print $1}' | grep -qx monitor; then
    ok "monitor module is already loaded."
else
    insmod ./monitor.ko
    INSERTED_BY_SCRIPT=1
    ok "insmod monitor.ko succeeded."
fi

for _ in $(seq 1 10); do
    [[ -e /dev/container_monitor ]] && break
```

```
    sleep 0.2
done
[[ -e /dev/container_monitor ]] || fail "/dev/container_monitor not found after module load."
ok "/dev/container_monitor exists."

if [[ "$INSERTED_BY_SCRIPT" -eq 1 ]]; then
    rmmod monitor
    INSERTED_BY_SCRIPT=0
    ok "rmmod monitor succeeded."
fi

echo "Preflight passed."
```

---

5. RESULT AND DISCUSSION

The experiments conducted using the container runtime provided clear insights into Linux process management and scheduling behavior. In the priority-based experiment with two CPU-bound processes, it was observed that the process with a lower nice value (higher priority) consistently received a greater share of CPU time. This resulted in faster execution compared to the process with a higher nice value. Monitoring through the `top` command showed a noticeable difference in CPU usage percentages, confirming that priority directly influences CPU allocation.

In the mixed workload experiment, where a CPU-bound process and an I/O-bound process were executed concurrently, distinct behavior was observed. The CPU-bound process continuously utilized the processor and maintained high CPU usage. In contrast, the I/O-bound process, which included periodic `sleep()` calls, executed intermittently and showed lower CPU usage. However, it remained responsive and produced output at regular intervals. This indicates that the scheduler quickly schedules I/O-bound processes when they become ready, ensuring responsiveness.

The results demonstrate the working of the Linux Completely Fair Scheduler (CFS), which aims to balance fairness and efficiency. CFS tracks the virtual runtime of processes and allocates CPU time such that no process is starved. CPU-bound processes tend to accumulate runtime faster due to continuous execution, while I/O-bound processes, which frequently wait, are given quicker access to the CPU upon waking. This behavior improves overall system responsiveness without significantly affecting throughput.

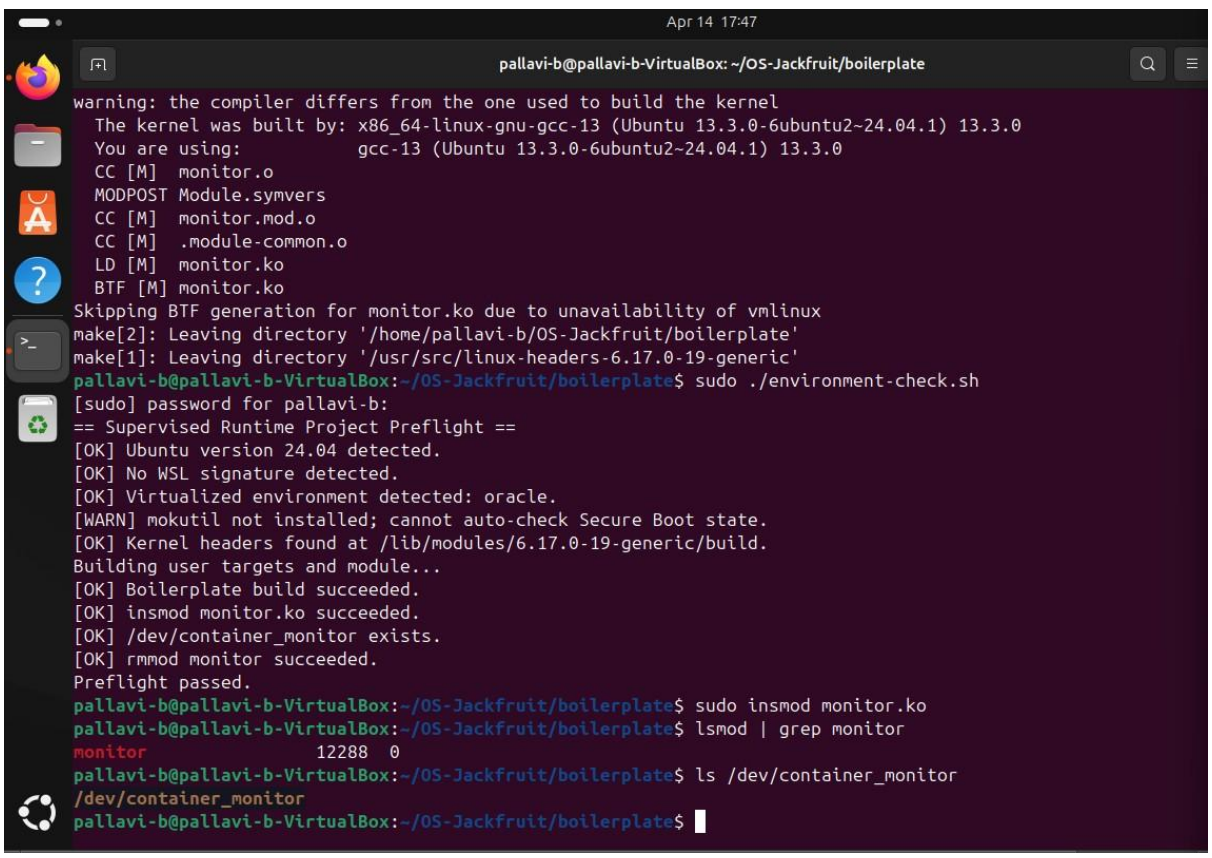
Additionally, the integration of the container runtime allowed controlled execution of multiple workloads, making it easier to observe scheduling patterns and system behavior. The use of system monitoring tools and logs provided measurable evidence of CPU usage and execution patterns.

Overall, the experiments validate that Linux scheduling effectively balances CPU utilization and responsiveness. The results align with theoretical expectations of process priority handling and CFS behavior, demonstrating fairness, efficiency, and adaptability in multi-process environments.

---

6. OUTPUTS

Task 1



```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate/rootfs$ ls
alpine-minirootfs-3.23.3-x86_64.tar.gz  dev  home  media  opt   root  sbin  sys  usr
bin                                     etc  lib   mnt    proc  run   srv   tmp  var
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate/rootfs$ cd ..
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo cp memory_hog cpu_hog io_pulse rootfs/
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ ls rootfs/
alpine-minirootfs-3.23.3-x86_64.tar.gz  cpu_hog  etc   io_pulse  media       mnt  proc  run   srv  tmp  var
bin                                     dev      home  lib       memory_hog  opt  root  sbin  sys  usr
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine supervisor rootfs
[supervisor] ready (rootfs=rootfs socket=/tmp/mini_runtime.sock)
[supervisor] started 'c1' pid=9289
[supervisor] started 'c2' pid=9340
```

```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine start c1 rootfs "cpu_hog 20"
[sudo] password for pallavi-b:
ok
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine start c2 rootfs "cpu_hog 20"
ok
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
c2               pid=9340   state=exited   soft= 40MiB hard= 64MiB code=127 sig=0   start=18:00:26
c1               pid=9289   state=exited   soft= 40MiB hard= 64MiB code=127 sig=0   start=17:59:40
```

```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine start c4 rootfs "/cpu_hog 100"
ok
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
c4               pid=15111  state=running  soft= 40MiB hard= 64MiB code=0   sig=0   start=18:17:11
c3               pid=11641  state=exited   soft= 40MiB hard= 64MiB code=0   sig=0   start=18:15:28
c2               pid=10204  state=exited   soft= 40MiB hard= 64MiB code=0   sig=0   start=18:08:35
c1               pid=10180  state=exited   soft= 40MiB hard= 64MiB code=0   sig=0   start=18:08:25
c2               pid=9340   state=exited   soft= 40MiB hard= 64MiB code=127 sig=0   start=18:00:26
c1
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Task 2

```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
c2               pid=10204  state=exited   soft= 40MiB hard= 64MiB code=0   sig=0   start
c1               pid=10180  state=exited   soft= 40MiB hard= 64MiB code=0   sig=0   start
c2               pid=9340   state=exited   soft= 40MiB hard= 64MiB code=127 sig=0   start
c1               pid=9289   state=exited   soft= 40MiB hard= 64MiB code=127 sig=0   start

pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ ls   rootfs/
alpine-minirootfs-3.23.3-x86_64.tar.gz  cpu_hog  etc   io_pulse  media       mnt  proc  r
bin                                     dev      home  lib       memory_hog  opt  root  s
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine logs c1
/bin/sh: cpu_hog: not found
cpu_hog alive elapsed=1 accumulator=8488015175725468995
cpu_hog alive elapsed=2 accumulator=6033449358966712512
cpu_hog alive elapsed=3 accumulator=4339841198134536119
cpu_hog alive elapsed=4 accumulator=7292795584196533636
cpu_hog alive elapsed=5 accumulator=8615908445189209954
cpu_hog alive elapsed=6 accumulator=2397641973987662503
cpu_hog alive elapsed=7 accumulator=17675932111116908633
cpu_hog alive elapsed=8 accumulator=12075446813871187839
cpu_hog alive elapsed=9 accumulator=11390401373559937977
cpu_hog alive elapsed=10 accumulator=6427755514765479242
cpu_hog alive elapsed=11 accumulator=5547866628412097498
cpu_hog alive elapsed=12 accumulator=13225349365678508617
cpu_hog alive elapsed=13 accumulator=14462013591419234240
cpu_hog done duration=20 accumulator=6801000933385675702
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine stop c1
container not found or not running
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$
```

Task 3

```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine run c3 rootfs "/cpu_hog 5"
[run] waiting for container to finish…
state=exited code=0 signal=0
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ cat logs/c3.log
cpu_hog alive elapsed=1 accumulator=6714212733155315941
cpu_hog alive elapsed=2 accumulator=6618240562122467652
cpu_hog alive elapsed=3 accumulator=16239123005603313596
cpu_hog alive elapsed=4 accumulator=951432212908341971
cpu_hog alive elapsed=5 accumulator=18109593245130871426
cpu_hog alive elapsed=6 accumulator=686108703922880681
cpu_hog alive elapsed=7 accumulator=3401053677491740527
cpu_hog alive elapsed=8 accumulator=12233038655658966537
cpu_hog alive elapsed=9 accumulator=2041230254743977408
cpu_hog alive elapsed=13 accumulator=7170637677605683577
cpu_hog alive elapsed=14 accumulator=16393574937059400447
cpu_hog alive elapsed=15 accumulator=11827388193847270057
cpu_hog alive elapsed=16 accumulator=14377626064443733320
cpu_hog alive elapsed=17 accumulator=13522037886052027350
cpu_hog alive elapsed=18 accumulator=7607683809324154061
cpu_hog alive elapsed=19 accumulator=12651013183571407044
cpu_hog alive elapsed=20 accumulator=2518993587427591787
cpu_hog alive elapsed=21 accumulator=4434451414469635023
cpu_hog alive elapsed=22 accumulator=13965148304984013561
```

```
Apr 14 18:21
pallavi-b@pallavi-b-VirtualBox: ~/OS-Jackfruit/boilerplate
pallavi-b@pallavi-b-VirtualBox: ~/OS-Jackfruit/boilerplate   pallavi-b@pallavi-b-VirtualBox: ~/OS-Jackfruit/boilerplate
cpu_hog alive elapsed=25 accumulator=6534811994822408123
cpu_hog alive elapsed=26 accumulator=6760259344068015841
cpu_hog alive elapsed=30 accumulator=11472209706401675 15
cpu_hog alive elapsed=31 accumulator=5364839943822199549
cpu_hog alive elapsed=32 accumulator=7264092236270666038
cpu_hog alive elapsed=33 accumulator=4499155384538955452
cpu_hog alive elapsed=34 accumulator=14639179011983481384
cpu_hog alive elapsed=35 accumulator=12865104147413858319
cpu_hog alive elapsed=36 accumulator=12712876961592311047
cpu_hog alive elapsed=37 accumulator=1128544247641671581
cpu_hog alive elapsed=38 accumulator=386858397204211678
cpu_hog alive elapsed=39 accumulator=6015178973589781073
cpu_hog alive elapsed=54 accumulator=15006476381075981262
cpu_hog alive elapsed=55 accumulator=17918464280488886155
cpu_hog alive elapsed=56 accumulator=632456040251411127
cpu_hog alive elapsed=57 accumulator=7384659404389094684
cpu_hog alive elapsed=58 accumulator=10968236714019063 27
cpu_hog alive elapsed=59 accumulator=9557075351983169063
cpu_hog alive elapsed=60 accumulator=411175023313787733
cpu_hog alive elapsed=61 accumulator=7619184934965498454
cpu_hog alive elapsed=62 accumulator=1502981503678653814
cpu_hog alive elapsed=63 accumulator=17685250712738112 90
cpu_hog alive elapsed=64 accumulator=5842404482700947076
cpu_hog alive elapsed=65 accumulator=4295718307275054603
cpu_hog alive elapsed=79 accumulator=13487021162472607042
cpu_hog alive elapsed=80 accumulator=10544385020702347777
cpu_hog alive elapsed=81 accumulator=1704060486756370775
cpu_hog alive elapsed=82 accumulator=13684449482333565960
cpu_hog alive elapsed=83 accumulator=3608759285278802025
cpu_hog alive elapsed=84 accumulator=11782575117011700673
cpu_hog alive elapsed=85 accumulator=14185679886525056786
```

Task 4

```
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo ./engine run m1 rootfs "/memory_hog 8 500"
[run] waiting for container to finish…
state=killed code=0 signal=9
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ dmesg | tail
dmesg: read kernel buffer failed: Operation not permitted
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$ sudo dmesg | tail
[ 4258.085869] [container_monitor] pid=16902 (container=c3) gone, removing entry
[ 4292.143385] workqueue: vmstat_update hogged CPU for >10000us 7 times, consider switching to WQ_UNBOUND
[ 4380.408484] [container_monitor] Registered container=m1 pid=17838 soft=41943040 hard=67108864
[ 4382.867403] [container_monitor] SOFT LIMIT container=m1 pid=17838 rss=42409984 limit=41943040
[ 4384.915134] [container_monitor] HARD LIMIT container=m1 pid=17838 rss=75964416 limit=67108864 - SIGKILL sent
[ 4393.101766] workqueue: delayed_vfree_work hogged CPU for >10000us 7 times, consider switching to WQ_UNBOUND
[ 4434.611571] audit: type=1400 audit(1776171183.890:414): apparmor="DENIED" operation="open" class="file" profile="snap
.firefox.firefox" name="/proc/pressure/memory" pid=4141 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=100
0 ouid=0
[ 4444.967648] audit: type=1400 audit(1776171194.251:415): apparmor="DENIED" operation="open" class="file" profile="snap
.firefox.firefox" name="/proc/pressure/memory" pid=4141 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=100
0 ouid=0
[ 4482.619376] audit: type=1400 audit(1776171231.904:416): apparmor="DENIED" operation="open" class="file" profile="snap
.firefox.firefox" name="/proc/pressure/memory" pid=4141 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=100
0 ouid=0
[ 4487.622521] audit: type=1400 audit(1776171236.907:417): apparmor="DENIED" operation="open" class="file" profile="snap
.firefox.firefox" name="/proc/pressure/memory" pid=4141 comm="MemoryPoller" requested_mask="r" denied_mask="r" fsuid=100
0 ouid=0
pallavi-b@pallavi-b-VirtualBox:~/OS-Jackfruit/boilerplate$
```

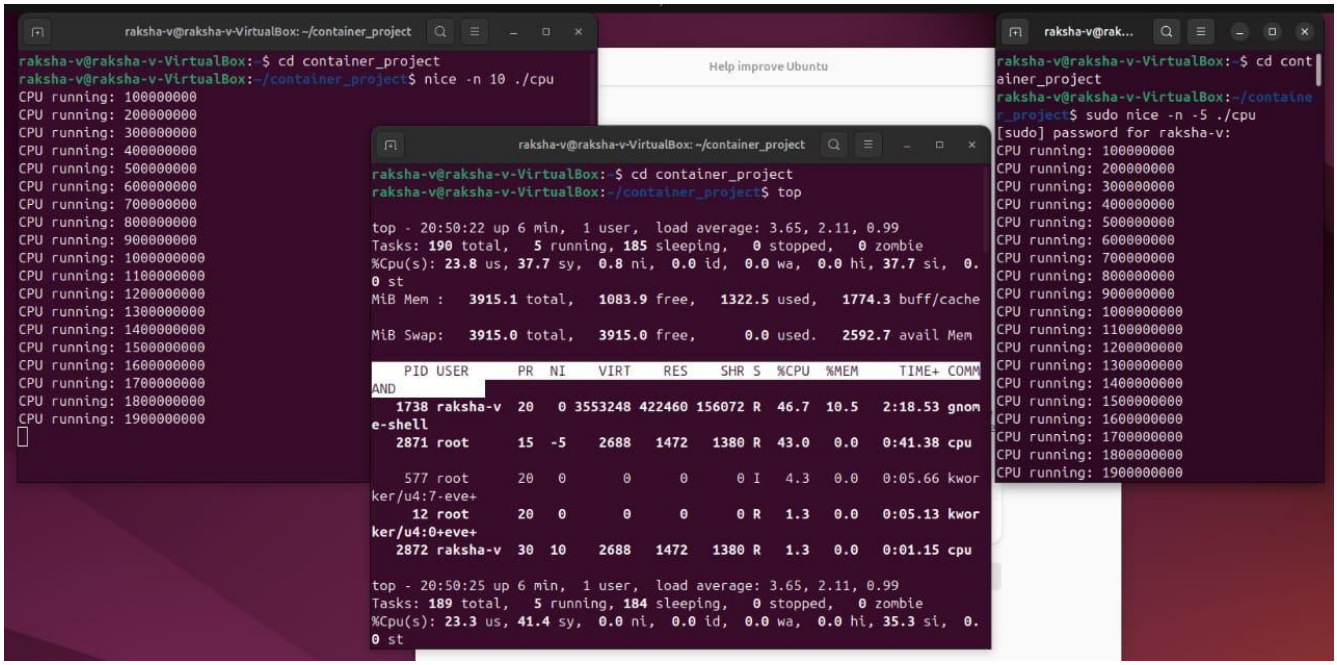
Task 5

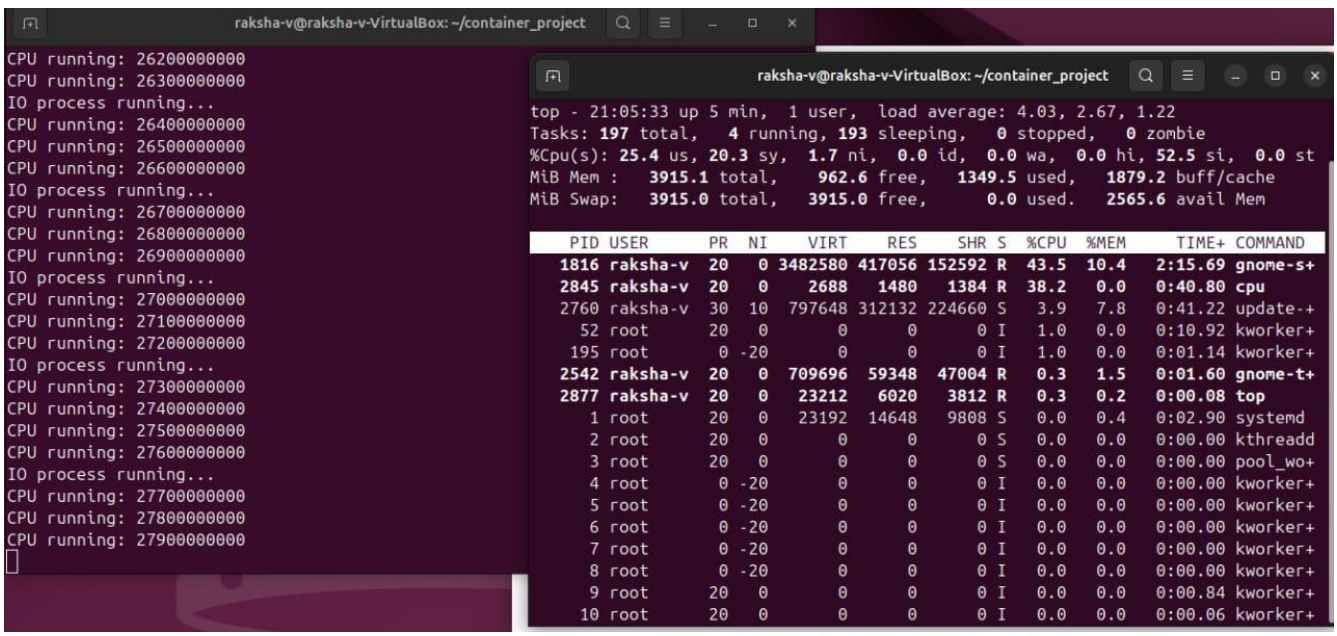
```
1234: command not found

[5]+  Stopped                 sudo nice -n -5 ./cpu
raksha-v@raksha-v-VirtualBox:~/container_project$ sudo nice -n -5 ./cpu
[sudo] password for raksha-v:
CPU running: 100000000
CPU running: 200000000
CPU running: 300000000
CPU running: 400000000
CPU running: 500000000
CPU running: 600000000
CPU running: 700000000
CPU running: 800000000
CPU running: 900000000
CPU running: 1000000000
CPU running: 1100000000
CPU running: 1200000000
CPU running: 1300000000
CPU running: 1400000000
CPU running: 1500000000
```

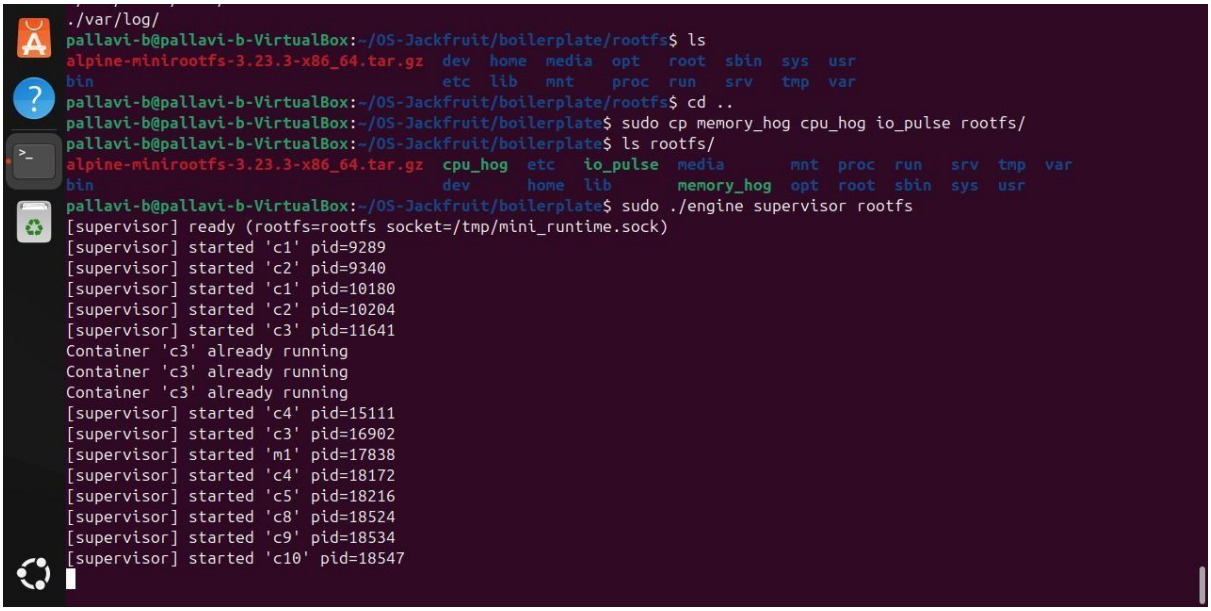
```
raksha-v@raksha-v-VirtualBox:~$ cd ~/container_project
raksha-v@raksha-v-VirtualBox:~/container_project$ nice -n 10 ./cpu
CPU running: 100000000
CPU running: 200000000
CPU running: 300000000
CPU running: 400000000
CPU running: 500000000
CPU running: 600000000
```

```
CPU running: 2500000000
CPU running: 2600000000
IO process running...
CPU running: 2700000000
IO process running...
CPU running: 2800000000
CPU running: 2900000000
IO process running...
CPU running: 3000000000
CPU running: 3100000000
CPU running: 3200000000
IO process running...
CPU running: 3300000000
CPU running: 3400000000
CPU running: 3500000000
IO process running...
CPU running: 3600000000
CPU running: 3700000000
CPU running: 3800000000
IO process running...
CPU running: 3900000000
CPU running: 4000000000
IO process running...
```


raksha-v@raksha-v-VirtualBox: ~/container_project
raksha-v@raksha-v-VirtualBox:~$ cd container_project
raksha-v@raksha-v-VirtualBox:~/container_project$ nice -n 10 ./cpu
CPU running: 100000000
CPU running: 200000000
CPU running: 300000000
CPU running: 400000000
CPU running: 500000000
CPU running: 600000000
CPU running: 700000000
CPU running: 800000000
CPU running: 900000000
CPU running: 1000000000
CPU running: 1100000000
CPU running: 1200000000
CPU running: 1300000000
CPU running: 1400000000
CPU running: 1500000000
CPU running: 1600000000
CPU running: 1700000000
CPU running: 1800000000
CPU running: 1900000000
Help improve Ubuntu
raksha-v@raksha-v-VirtualBox: ~/container_project
raksha-v@raksha-v-VirtualBox:~$ cd container_project
raksha-v@raksha-v-VirtualBox:~/container_project$ top
top - 20:50:22 up 6 min, 1 user, load average: 3.65, 2.11, 0.99
Tasks: 190 total, 5 running, 185 sleeping, 0 stopped, 0 zombie
%Cpu(s): 23.8 us, 37.7 sy, 0.8 ni, 0.0 id, 0.0 wa, 0.0 hi, 37.7 si, 0.0 st
MiB Mem : 3915.1 total, 1083.9 free, 1322.5 used, 1774.3 buff/cache
MiB Swap: 3915.0 total, 3915.0 free, 0.0 used. 2592.7 avail Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
1738 raksha-v 20 0 3553248 422460 156072 R 46.7 10.5 2:18.53 gnome-shell
2871 root 15 -5 2688 1472 1380 R 43.0 0.0 0:41.38 cpu
577 root 20 0 0 0 0 I 4.3 0.0 0:05.66 kworker/u4:7-eve+
12 root 20 0 0 0 0 R 1.3 0.0 0:05.13 kworker/u4:0+eve+
2872 raksha-v 30 10 2688 1472 1380 R 1.3 0.0 0:01.15 cpu
top - 20:50:25 up 6 min, 1 user, load average: 3.65, 2.11, 0.99
Tasks: 189 total, 5 running, 184 sleeping, 0 stopped, 0 zombie
%Cpu(s): 23.3 us, 41.4 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 35.3 si, 0.0 st
raksha-v@rak...
raksha-v@raksha-v-VirtualBox:~$ cd container_project
raksha-v@raksha-v-VirtualBox:~/container_project$ sudo nice -n -5 ./cpu
[sudo] password for raksha-v:
CPU running: 100000000
CPU running: 200000000
CPU running: 300000000
CPU running: 400000000
CPU running: 500000000
CPU running: 600000000
CPU running: 700000000
CPU running: 800000000
CPU running: 900000000
CPU running: 1000000000
CPU running: 1100000000
CPU running: 1200000000
CPU running: 1300000000
CPU running: 1400000000
CPU running: 1500000000
CPU running: 1600000000
CPU running: 1700000000
CPU running: 1800000000
CPU running: 1900000000


raksha-v@raksha-v-VirtualBox: ~/container_project
CPU running: 26200000000
CPU running: 26300000000
IO process running...
CPU running: 26400000000
CPU running: 26500000000
CPU running: 26600000000
IO process running...
CPU running: 26700000000
CPU running: 26800000000
CPU running: 26900000000
IO process running...
CPU running: 27000000000
CPU running: 27100000000
CPU running: 27200000000
IO process running...
CPU running: 27300000000
CPU running: 27400000000
CPU running: 27500000000
CPU running: 27600000000
IO process running...
CPU running: 27700000000
CPU running: 27800000000
CPU running: 27900000000
raksha-v@raksha-v-VirtualBox: ~/container_project
top - 21:05:33 up 5 min, 1 user, load average: 4.03, 2.67, 1.22
Tasks: 197 total, 4 running, 193 sleeping, 0 stopped, 0 zombie
%Cpu(s): 25.4 us, 20.3 sy, 1.7 ni, 0.0 id, 0.0 wa, 0.0 hi, 52.5 si, 0.0 st
MiB Mem : 3915.1 total, 962.6 free, 1349.5 used, 1879.2 buff/cache
MiB Swap: 3915.0 total, 3915.0 free, 0.0 used. 2565.6 avail Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
1816 raksha-v 20 0 3482580 417056 152592 R 43.5 10.4 2:15.69 gnome-s+
2845 raksha-v 20 0 2688 1480 1384 R 38.2 0.0 0:40.80 cpu
2760 raksha-v 30 10 797648 312132 224660 S 3.9 7.8 0:41.22 update-+
52 root 20 0 0 0 0 I 1.0 0.0 0:10.92 kworker+
195 root 0 -20 0 0 0 I 1.0 0.0 0:01.14 kworker+
2542 raksha-v 20 0 709696 59348 47004 R 0.3 1.5 0:01.60 gnome-t+
2877 raksha-v 20 0 23212 6020 3812 R 0.3 0.2 0:00.08 top
1 root 20 0 23192 14648 9808 S 0.0 0.4 0:02.90 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.00 pool_wo+
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker+
5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker+
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker+
7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker+
8 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker+
9 root 20 0 0 0 0 I 0.0 0.0 0:00.84 kworker+
10 root 20 0 0 0 0 I 0.0 0.0 0:00.06 kworker+

Task 6



GITHUB LINK:

https://github.com/Raksha269/OS-Jackfruit